

Universidade de São Paulo

Bacharelado em Engenharia de Software

Heverton Eduardo Peres

**AI Driven Development: Engenharia de Software Dirigida por
Agentes de Inteligência Artificial**

São Paulo
2026

Heverton Eduardo Peres

**AI Driven Development: Engenharia de Software Dirigida por
Agentes de Inteligência Artificial**

Trabalho de Conclusão de Curso apresentado ao curso de
Bacharelado em Engenharia de Software como requisito
parcial para obtenção do título de graduado.

Orientador(a): Prof. Dr. Orientador

São Paulo
2026

Heverton Eduardo Peres

**AI Driven Development: Engenharia de Software Dirigida por Agentes de
Inteligência Artificial**

Trabalho de Conclusão de Curso aprovado como requisito parcial para obtenção do título de graduado,
pela banca examinadora formada por:

Membro da banca examinadora

Membro da banca examinadora

Membro da banca examinadora

São Paulo, 2026

RESUMO

O AI Driven Development (ADD) consolida-se como paradigma emergente de engenharia de software no qual modelos de linguagem de grande escala e agentes de inteligência artificial participam ativamente de todas as fases do ciclo de vida do software, da especificação à manutenção. Esta pesquisa, de caráter qualitativo-exploratório baseada em revisão da literatura científica e técnica, investigou os fundamentos do paradigma, a evolução dos modelos de linguagem e da geração de código, a arquitetura de agentes autônomos, os sistemas multiagentes e protocolos de interoperabilidade, o ecossistema de ferramentas, os benchmarks de avaliação, e as dimensões de qualidade, segurança e dívida técnica do código gerado. Os resultados indicam que a evidência de ganho de produtividade é robusta, com aumento médio de 26,08% nas tarefas concluídas por semana documentado em estudo randomizado controlado, porém concentrada em tarefas bem delimitadas; que benchmarks de segurança documentam código inseguro em 35% a 65% dos casos, agravado pelo paradoxo da confiança; e que o código gerado acumula dívida técnica estrutural identificável, descrita como assinatura de máquina. Conclui-se que o sucesso da adoção de IA na engenharia de software depende menos da capacidade dos modelos e mais da capacidade das organizações de orquestrar agentes com guardrails de qualidade, segurança e governança, incluindo análise estática obrigatória, revisão humana e medição contínua.

Palavras-chave: Inteligência Artificial; Engenharia de Software; Agentes Autônomos; Geração de Código; Qualidade de Software.

ABSTRACT

AI Driven Development (ADD) is consolidating as an emerging software engineering paradigm in which large language models and artificial intelligence agents actively participate in every phase of the software lifecycle, from specification to maintenance. This qualitative-exploratory research, based on a review of scientific and technical literature, investigated the paradigm's foundations, the evolution of language models and code generation, autonomous agent architectures, multi-agent systems and interoperability protocols, the tooling ecosystem, evaluation benchmarks, and the quality, security, and technical debt dimensions of generated code. Results indicate that the evidence for productivity gains is robust, with an average 26.08% increase in tasks completed per week documented in a randomized controlled trial, though concentrated in well-delimited tasks; that security benchmarks document insecure code in 35% to 65% of cases, aggravated by the trust paradox; and that generated code accumulates identifiable structural technical debt, described as a machine signature. It concludes that the success of AI adoption in software engineering depends less on model capability and more on organizations' ability to orchestrate agents with quality, security, and governance guardrails, including mandatory static analysis, human review, and continuous measurement.

Keywords: Artificial Intelligence; Software Engineering; Autonomous Agents; Code Generation; Software Quality.

SUMÁRIO

1	Introdução	9
1.1	Contextualização da problemática	9
1.2	O problema de pesquisa	10
1.3	Objetivos	10
1.3.1	Objetivo geral	10
1.3.2	Objetivos específicos	11
1.4	Justificativa	11
1.5	Metodologia	12
1.6	Estrutura do trabalho	12
		13
2	Fundamentos da Engenharia de Software Dirigida por IA	14
2.1	Contextualização: a natureza da mudança de paradigma	14
2.2	Referencial Teórico: taxonomia dos paradigmas de LLM na engenharia de software	14
2.2.1	Os três paradigmas de GUO et al. (2025)	14
2.2.2	Agentes de software: definição e capacidades	15
2.2.3	Interface agente-computador (ACI)	15
2.2.4	O ciclo de vida do software impactado	15
2.3	Análise: da assistência reativa à autonomia proativa	16
2.4	Síntese Parcial	16
		18
3	Modelos de Linguagem e Geração de Código	19
3.1	Contextualização	19
3.2	Referencial Teórico: evolução dos modelos de código	19
3.2.1	Do Codex aos modelos agênticos	19
3.2.2	Aprendizagem de uso de ferramentas	20
3.2.3	Benchmarks funcionais: HumanEval e sucessores	20
3.3	Análise: capacidades e limitações da geração de código	20
3.3.1	A qualidade funcional é condição necessária, mas insuficiente	20
3.3.2	Segurança do código gerado	20
3.3.3	O paradoxo da confiança	21
3.3.4	Manutenibilidade e dívida técnica	21
3.3.5	Heterogeneidade dos resultados	21
3.4	Síntese Parcial	21
		23
4	Agentes de Software Autônomos	24

4.1	Contextualização	24
4.2	Referencial Teórico: arquitetura dos agentes	24
4.2.1	A Interface Agente-Computador (ACI)	24
4.2.2	Loops de raciocínio-ação	24
4.2.3	Memória e ferramentas	25
4.3	Análise: sistemas de referência	25
4.3.1	SWE-agent	25
4.3.2	OpenHands	25
4.3.3	Devin e os agentes gerenciados em nuvem	25
4.3.4	Abordagens alternativas e o debate da complexidade	26
4.4	Síntese Parcial	26
		27
5	Sistemas Multiagentes e Protocolos de Interoperabilidade	28
5.1	Do agente isolado à colaboração entre agentes	28
5.2	Arquiteturas de orquestração: o caso MetaGPT	28
5.3	Ciclos de reflexão e autocrítica entre agentes	29
5.4	Protocolos de interoperabilidade entre agentes	29
5.5	Padrões emergentes e implicações para as equipes de desenvolvimento	30
5.6	Síntese parcial	31
		32
6	Ecossistema de Ferramentas e Práticas de Mercado	33
6.1	Panorama histórico e consolidação do mercado	33
6.2	Assistentes de IDE: a camada ubíqua	33
6.3	Agentes em nuvem: execução assíncrona e paralelismo	34
6.4	Agentes locais e síncronos: controle e contexto profundo	34
6.5	Práticas de adoção: do piloto à governança	34
6.6	Síntese parcial	35
		36
7	Avaliação de Agentes: Benchmarks e Lacunas de Medição	37
7.1	A necessidade de avaliação padronizada	37
7.2	O SWE-bench e o SWE-bench Verified	37
7.3	Outros benchmarks e a medição de habilidades específicas	38
7.4	Lacunas metodológicas dos benchmarks atuais	38
7.5	Rumos da avaliação: do benchmark à medição organizacional	39
7.6	Síntese parcial	39
		40
8	Qualidade, Segurança e Dívida Técnica do Código Gerado	41
8.1	Corretude funcional versus qualidade estrutural	41
8.2	Insegurança por padrão: evidências de benchmarks de segurança	41
8.3	Dívida técnica e a “assinatura de máquina”	42
8.4	Guardrails organizacionais e técnicos	42

8.5 Síntese parcial	43
.....	44
9 Produtividade e Impacto Organizacional da Adoção de IA	45
9.1 Evidência experimental de ganhos de produtividade	45
9.2 Onde a produtividade aumenta e onde não muda	45
9.3 Impacto sobre habilidades e dinâmica de equipes	46
9.4 Riscos organizacionais e condições de sucesso	46
9.5 Síntese parcial	47
.....	48
10 Considerações Finais	49
10.1 Síntese da pesquisa	49
10.2 Principais achados	49
10.3 Implicações práticas	50
10.4 Limitações da pesquisa	50
10.5 Desdobramentos futuros	50
10.6 Conclusão	51
.....	52
Referências	53

1 INTRODUÇÃO

1.1 Contextualização da problemática

A engenharia de software, desde a sua consolidação como disciplina nas décadas de 1960 e 1970, caracteriza-se pela busca contínua de métodos, processos e ferramentas que elevem a produtividade e a qualidade dos artefatos produzidos. Essa trajetória é marcada por sucessivas ondas de automação — dos compiladores às linguagens de alto nível, dos ambientes integrados de desenvolvimento (IDEs) às plataformas de integração contínua —, cada uma das quais redistribuiu as tarefas entre humanos e máquinas (STAHNKE; VAHLICK, 2013). A mais recente dessas ondas, protagonizada pelos modelos de linguagem de grande escala (large language models — LLMs) e pelos agentes de inteligência artificial, inaugura um paradigma que a literatura recente passou a denominar *AI Driven Development* (ADD): um modelo de produção de software em que modelos de linguagem e agentes participam ativamente de todas as fases do ciclo de vida — especificação, arquitetura, codificação, testes, revisão, implantação e manutenção —, atuando de forma reativa, proativa ou autônoma (TEQNOVOS, 2025; BAYTECH CONSULTING, 2026).

O marco fundacional dessa trajetória é o lançamento, em 2021, do Codex original, modelo treinado em código que demonstrou a viabilidade de gerar programas completos a partir de descrições em linguagem natural (CHEN et al., 2021). No mesmo período, a padronização de benchmarks funcionais como HumanEval e MBPL estabeleceu um regime de avaliação da geração de código que se tornou referência para os estudos subsequentes (CHEN et al., 2021). Nos anos seguintes, a evolução deixou de se limitar à geração pontual de trechos de código e passou a abranger sistemas agênticos capazes de planejar, navegar em repositórios, editar arquivos, executar comandos e validar resultados de forma iterativa — os chamados agentes de software (software engineering agents — SWE agents) (YANG et al., 2024; DONG et al., 2025).

A relevância do fenômeno é quantificável. Estudos controlados randomizados realizados com 4.867 desenvolvedores em empresas como Microsoft, Accenture e organizações da Fortune 100 identificaram ganho médio de 26,08% nas tarefas concluídas por semana entre usuários de assistentes de codificação (CUI et al., 2025). O relatório DORA de 2025 indica que cerca de 90% dos respondentes utilizam inteligência artificial generativa no trabalho diário e que 80% se percebem mais produtivos com o seu uso (DORA/GOOGLE CLOUD, 2025). No plano macro, estima-se que Google e Microsoft já tenham cerca de 30% do código novo escrito ou assistido por IA (ALURA, 2026), enquanto o GitHub Octoverse registra crescimento ano a ano na parcela de código gerado ou assistido por modelos (FUTUREWARN, 2026).

1.2 O problema de pesquisa

Apesar da rápida adoção, o conhecimento sobre as reais capacidades, limites e consequências do desenvolvimento dirigido por IA permanece fragmentado e, em vários pontos, contraditório. De um lado, os resultados em benchmarks públicos de resolução de issues reais, como o SWE-bench Verified, alcançam índices crescentes de acerto em subconjuntos validados por humanos (JIMENEZ et al., 2024; LLM-STATS, 2026). De outro, evidências empíricas indicam que o desempenho em benchmarks não se transfere linearmente para o trabalho real de engenharia: tarefas que envolvem coordenação multi-semana, revisão entre pares e decisões de produto não são capturadas pelos cenários sintéticos (BENCHMARKING AGENTS, 2026). Estudos de segurança demonstram que modelos líderes produzem código inseguro em 35% a 40% das vezes (BHATTAHALI et al., 2024) e que desenvolvedores que usam assistentes de IA tendem a escrever código menos seguro ao mesmo tempo em que acreditam mais em sua segurança — fenômeno denominado *paradoxo da confiança* (PERRY et al., 2022; SNYK, 2025).

No âmbito organizacional, o relatório DORA de 2025 estabelece que a IA é um amplificador de desempenho, e não uma solução: a adoção correlaciona-se com maior throughput, mas também com maior instabilidade, mais falhas de mudança e retrabalho quando não há sistemas e processos maduros (DORA/GOOGLE CLOUD, 2025; INFOQ, 2026). Paralelamente, pesquisas sobre a qualidade estrutural do código gerado identificam uma “assinatura de máquina” na dívida técnica — padrões como bloat procedural, God Classes e acoplamento cíclico —, que persistem mesmo em código funcionalmente correto (ZHU; TSANTALIS; RIGBY, 2026).

Configura-se, portanto, um problema de pesquisa com dupla face: por um lado, compreender o que o paradigma ADD é capaz de realizar na prática; por outro, identificar os mecanismos de avaliação, governança e garantia de qualidade que determinam se essa adoção produz ganhos sustentáveis ou amplifica disfunções existentes. A literatura oferece revisões sistemáticas abrangentes sobre agentes em engenharia de software (JIN et al., 2024; LIU et al., 2024; JIANG; LO; LIU, 2025), mas a integração dos achados de benchmarks, estudos de produtividade, segurança e qualidade estrutural em uma síntese coesa, em língua portuguesa e orientada à realidade das organizações de software, permanece uma lacuna.

1.3 Objetivos

1.3.1 Objetivo geral

Analisar o paradigma do desenvolvimento de software dirigido por agentes de inteligência artificial — o *AI Driven Development* —, caracterizando seus fundamentos, ferramentas, formas de avaliação, riscos e implicações para a engenharia de software da atualidade.

1.3.2 Objetivos específicos

- a) Descrever os fundamentos conceituais do ADD, incluindo a taxonomia dos paradigmas prompt-based, fine-tune-based e agent-based da engenharia de software impulsionada por LLMs;
- b) Examinar a evolução dos modelos de linguagem aplicados à geração de código e os instrumentos de avaliação funcional dessa geração;
- c) Analisar a arquitetura dos agentes de software autônomos, seus loops de raciocínio, memória, uso de ferramentas e interfaces agente-computador;
- d) Investigar os sistemas multiagentes e os protocolos de interoperabilidade que sustentam a infraestrutura de desenvolvimento agêntico;
- e) Mapear o ecossistema de ferramentas comerciais e as práticas emergentes de adoção, incluindo o desenvolvimento dirigido por especificação;
- f) Avaliar criticamente os benchmarks de agentes e as lacunas metodológicas de medição, em especial o hiato entre desempenho em benchmark e desempenho em produção;
- g) Sintetizar as evidências sobre qualidade, segurança e dívida técnica do código gerado por LLMs e agentes;
- h) Analisar as evidências empíricas de produtividade e os modelos organizacionais de adoção de IA, com ênfase no relatório DORA de 2025.

1.4 Justificativa

A justificativa desta pesquisa assenta-se em três dimensões complementares. A primeira é de ordem prática: organizações de todos os portes estão incorporando ferramentas de IA ao fluxo de desenvolvimento sem dispor de evidências organizadas sobre como fazê-lo de forma segura e produtiva (ALURA, 2026; FUTUREWARNS, 2026). A compreensão dos guardrails de qualidade — revisão humana obrigatória, análise estática de segurança (SAST), varredura de dependências (SCA) e governança de processos — é condição para que os ganhos documentados nos estudos controlados (CUI et al., 2025) não sejam convertidos em débito técnico e vulnerabilidades (SNYK, 2025; ENDOR LABS, 2025).

A segunda dimensão é acadêmica. O campo é jovem e evolui rapidamente: os principais surveys de agentes para engenharia de software foram publicados entre 2024 e 2025 (JIN et al., 2024; LIU et al., 2024; GUO et al., 2025; JIANG; LO; LIU, 2025) e a literatura sobre a qualidade do código gerado encontra-se em consolidação (ZHU; TSANTALIS; RIGBY, 2026; SANTA MOLISON et al., 2025). A síntese crítica desses materiais em língua portuguesa contribui para a formação de engenheiros de software e para a redução da assimetria informacional entre a comunidade internacional e a brasileira.

A terceira dimensão é estratégica. Os relatórios setoriais projetam a consolidação do trabalho agêntico e da orquestração multiagente como direção dominante do desenvolvimento

de software (FORRESTER, 2026; TEQNOVOS, 2025). Compreender antecipadamente as capacidades, as limitações e as condições organizacionais de sucesso desse paradigma é requisito para a tomada de decisão informada por parte de gestores, arquitetos e desenvolvedores.

1.5 Metodologia

esta pesquisa caracteriza-se como qualitativa, de natureza exploratória-descritiva, baseada em revisão de literatura do tipo narrativa estruturada. O corpus foi constituído por quatro categorias de fontes: (i) artigos científicos revisados por pares e preprints indexados em arXiv e periódicos, cobrindo o período de 2013 a 2026; (ii) relatórios técnicos institucionais, com destaque para o DORA Report 2025 (DORA/GOOGLE CLOUD, 2025), o SWE-bench e seus derivados (JIMENEZ et al., 2024) e a especificação do Model Context Protocol (MODEL CONTEXT PROTOCOL, 2025); (iii) documentação e materiais oficiais de ferramentas e plataformas comerciais (TOOLBOXKART, 2026; OFLIGHT, 2026); e (iv) publicações técnicas da indústria, como análises do Snyk (SNYK, 2025) e da Endor Labs (ENDOR LABS, 2025).

A coleta foi realizada por meio de busca sistematizada em bases acadêmicas e repositórios técnicos, seguida de triagem por relevância temática e atualidade. O material foi organizado em um dossiê de pesquisa estruturado em seis blocos — conceitos-chave, artigos científicos, estado da arte, casos de uso corporativos, limitações e controvérsias, e fontes brutas —, posteriormente indexado para recuperação semântica. A análise seguiu os quatro momentos do framework ACAD: contextualização de cada tema, levantamento do referencial teórico pertinente, análise crítica das evidências e síntese parcial ao final de cada seção. A redação observou as normas ABNT, com numeração progressiva (NBR 6024) e citação autor-data (NBR 10520).

Delimita-se o escopo: não são objeto deste trabalho a comparação exaustiva de modelos de linguagem individuais, o desenvolvimento de novos benchmarks ou a realização de experimentos controlados com ferramentas específicas, embora tais elementos sejam mobilizados como evidência secundária. A análise se concentra no paradigma ADD como objeto de estudo, com recorte temporal de 2021 a 2026.

1.6 Estrutura do trabalho

O trabalho organiza-se em dez seções. A seção 2 estabelece os fundamentos da engenharia de software dirigida por IA, com a taxonomia dos paradigmas e o impacto sobre o ciclo de vida do software. A seção 3 analisa os modelos de linguagem e a geração de código. A seção 4 examina os agentes de software autônomos. A seção 5 aborda os sistemas multiagentes e os protocolos de interoperabilidade. A seção 6 mapeia o ecossistema de ferramentas e práticas de mercado. A seção 7 avalia os benchmarks de agentes e suas lacunas metodológicas. A seção 8 sintetiza as evidências sobre qualidade, segurança e dívida técnica. A seção 9 analisa produtividade e impacto organizacional. A seção 10 apresenta as considerações finais, limitações e direções futuras.



2 FUNDAMENTOS DA ENGENHARIA DE SOFTWARE DIRIGIDA POR IA

2.1 Contextualização: a natureza da mudança de paradigma

A engenharia de software constitui um campo historicamente dependente da intensidade de trabalho humano qualificado para a transformação de requisitos em artefatos executáveis. A emergência dos modelos de linguagem de grande escala introduz um ator novo nesse processo: uma entidade capaz de produzir artefatos textuais — e, portanto, código — com fluência próxima à humana, em escala e velocidade sem precedentes (CHEN et al., 2021; STAHNKE; VAHLICK, 2013). O que distingue a onda atual de automações anteriores é o deslocamento do locus da automação: não se automatiza mais apenas a compilação ou o gerenciamento de builds, mas a própria atividade de escrita de especificações, código, testes e documentação (SAUVOLA et al., 2024; TERRAGNI; ROOP; BLINCOE, 2024).

O termo *AI Driven Development* (ADD) foi cunhado para descrever esse paradigma: um modo de produção de software no qual LLMs e agentes de IA participam ativamente de todas as fases do ciclo de vida — especificação, arquitetura, codificação, testes, revisão, implantação e manutenção —, em graus variáveis de autonomia, de sugestões reativas até operação totalmente autônoma (TEQNOVOS, 2025; BAYTECH CONSULTING, 2026). Nessa definição, a IA deixa de ser uma ferramenta auxiliar pontual e passa a integrar a estrutura do processo, modificando papéis, competências e formas de colaboração dentro das equipes (STRAY et al., 2025; VINICIUS3W, 2025).

2.2 Referencial Teórico: taxonomia dos paradigmas de LLM na engenharia de software

2.2.1 Os três paradigmas de GUO et al. (2025)

A literatura consolidou a classificação da engenharia de software impulsionada por LLMs (*LLM-empowered software engineering*) em três paradigmas complementares (GUO et al., 2025):

- a) **Paradigma baseado em prompts (prompt-based):** o modelo é utilizado como um gerador de artefatos a partir de descrições textuais elaboradas por humanos. É o modo mais difundido, exemplificado por assistentes de autocompletar e chatbots de apoio à codificação (CHEN et al., 2021; STRAY et al., 2025). A qualidade da saída depende criticamente da qualidade

da entrada, o que impulsionou o desenvolvimento de técnicas de engenharia de prompt e de context engineering (MURUGESAN, 2026).

- b) **Paradigma baseado em ajuste fino (fine-tune-based):** modelos genéricos são especializados em tarefas de engenharia de software por meio de treinamento adicional sobre corpora de código e dados técnicos. Essa abordagem produz modelos de domínio, como os especializados em geração de código, capazes de incorporar convenções, idiomas de programação e padrões arquiteturais específicos (GUO et al., 2025; DONG et al., 2025).
- c) **Paradigma baseado em agentes (agent-based):** LLMs são incorporados a sistemas autônomos que planejam, executam ações sobre o ambiente (edição de arquivos, execução de comandos, consulta a ferramentas), observam os resultados e iteram até atingir um objetivo — os chamados *software engineering agents* (YANG et al., 2024; LIU et al., 2024; JIN et al., 2024).

2.2.2 Agentes de software: definição e capacidades

Um agente de software é definido como um sistema baseado em LLM que planeja, navega em repositórios de código, edita arquivos, executa comandos e valida resultados de forma iterativa para resolver tarefas reais de engenharia (YANG et al., 2024). Diferentemente dos assistentes de autocompletar, o agente opera em ciclos percepção-raciocínio-ação: interpreta o estado do repositório, decide o próximo passo, executa a ação e observa o efeito antes de prosseguir (YAO et al., 2023). Essa arquitetura de loop é o coração do paradigma agêntico e a fonte de sua capacidade de lidar com tarefas de composição — problemas que exigem múltiplas edições coordenadas em arquivos distintos (JIANG; LO; LIU, 2025; XIA et al., 2025).

2.2.3 Interface agente-computador (ACI)

A interação do agente com o ambiente computacional exige interfaces projetadas especificamente para modelos de linguagem. O conceito de *Agent-Computer Interface* (ACI) designa o conjunto de canais pelos quais o agente observa e age sobre o sistema — terminal, editor, navegador, APIs — e um dos achados mais relevantes do campo é que o desenho dessa interface impacta o desempenho do agente mais do que a escolha do modelo subjacente (YANG et al., 2024; CODESOTA, 2026). Interfaces com observações concisas, ações de alto nível e feedback estruturado reduzem a carga cognitiva do modelo e aumentam a taxa de sucesso na resolução de issues (YANG et al., 2024).

2.2.4 O ciclo de vida do software impactado

A participação de LLMs e agentes estende-se por todo o ciclo de vida: na especificação, com a elaboração assistida de requisitos e critérios de aceite; na arquitetura, com a geração de propostas de projeto e análise de trade-offs; na codificação, com a geração, refatoração e reparo de código; nos testes, com a síntese de casos de teste e a análise de cobertura; na revisão, com a

inspeção automatizada de pull requests; e na manutenção, com a triagem de issues e a correção de defeitos (GUO et al., 2025; TEQNOVOS, 2025; BAYTECH CONSULTING, 2026). Esse espectro completo faz do ADD um paradigma transversal, e não uma técnica pontual de geração de código.

2.3 Análise: da assistência reativa à autonomia proativa

A literatura documenta uma progressão contínua no grau de autonomia das ferramentas. O GitHub Copilot, lançado em 2021 sobre o modelo Codex original, inaugurou a assistência reativa de autocompletar (CHEN et al., 2021; TOOLBOXKART, 2026). A partir de 2023, padrões de *agentic coding* — em que o modelo decide a sequência de ações, planeja, usa ferramentas e reflete sobre resultados — passaram a dominar o discurso e a prática (YAO et al., 2023; SHINN et al., 2023; MINDSHARE, 2026). Em 2025, surgiram os primeiros agentes de codificação autônomos em nuvem, capazes de executar tarefas completas de forma assíncrona e paralela, criando pull requests sem intervenção humana contínua (TOOLBOXKART, 2026; AICOOLES, 2026).

Essa progressão não é meramente incremental: ela altera a distribuição de responsabilidade entre humano e máquina. No modo assistido, o desenvolvedor permanece o autor, e a IA sugere; no modo agêntico, o agente torna-se executor, e o desenvolvedor assume papel de supervisor e avaliador de saídas (STRAY et al., 2025; VINICIUS3W, 2025). Estudos de campo documentam a mudança percebida de papel: os desenvolvedores gastam tempo adicional verificando sugestões e relatam limitações de explicabilidade das decisões dos agentes (STRAY et al., 2025; STRAY; MOE; GANESHAN; KOBENES, 2025).

A análise crítica revela também uma tensão estrutural: quanto maior a autonomia concedida ao agente, maior a dependência da qualidade dos processos de verificação circundantes. Os dados de segurança são ilustrativos: mesmo os modelos líderes produzem código inseguro em parcela expressiva das amostras (BHATTACHALI et al., 2024; SCHERMANN et al., 2024), e a ausência de revisão humana sistemática converte a autonomia em risco (SNYK, 2025; ALURA, 2026). O ADD não elimina, portanto, a necessidade de engenharia — transfere-a para as camadas de especificação, verificação e governança.

2.4 Síntese Parcial

Os fundamentos do ADD revelam um paradigma em três camadas: uma camada de modelos (LLMs generalistas e especializados), uma camada de interfaces (ACIs e ferramentas) e uma camada de processos (fluxos de verificação e governança). A taxonomia de GUO et al. (2025) organiza os modos de uso, e a literatura agêntica demonstra que a eficácia depende tanto dos modelos quanto do desenho das interfaces e dos loops de ação (YANG et al., 2024; JIANG; LO; LIU, 2025). A mudança de papel do desenvolvedor — de autor para supervisor — é o efeito organizacional mais consistente documentado (STRAY et al., 2025). Esses fundamentos

fornece a base para as seções seguintes, que examinam os modelos e a geração de código (seção 3), os agentes autônomos (seção 4) e as camadas superiores do paradigma.



3 MODELOS DE LINGUAGEM E GERAÇÃO DE CÓDIGO

3.1 Contextualização

A geração de código por modelos de linguagem constitui o alicerce empírico do AI Driven Development. Desde os primeiros modelos capazes de completar trechos de programa até os sistemas agênticos atuais, a capacidade de produzir código funcional a partir de linguagem natural evoluiu de forma acelerada, impulsionada por avanços arquiteturais, por corpora de treinamento cada vez maiores e por regimes de avaliação cada vez mais rigorosos (CHEN et al., 2021; GUO et al., 2025). Esta seção analisa essa evolução, os instrumentos de medição e as capacidades e limitações documentadas da geração de código por LLMs.

3.2 Referencial Teórico: evolução dos modelos de código

3.2.1 Do Codex aos modelos agênticos

O marco inaugural da geração de código por LLMs em escala industrial foi o Codex, modelo treinado a partir do GPT-3 sobre corpus massivo de código público, apresentado em 2021 com o benchmark HumanEval (CHEN et al., 2021). O Codex demonstrou a viabilidade de resolver problemas de programação em linguagem natural, estabelecendo o paradigma de avaliação por pass@k e inspirando toda uma geração de assistentes, incluindo o GitHub Copilot (TOOLBOXKART, 2026). O modelo original foi depreciado em 2023, mas o nome persistiu na plataforma OpenAI Codex, relançada em 2025 como agente de codificação autônomo em nuvem (TOOLBOXKART, 2026; OFLIGHT, 2026).

A geração de código evoluiu em três frentes complementares. Na frente arquitetural, o surgimento de modelos especializados em código — como os da família Codex, os modelos com janelas de contexto estendidas e arquiteturas Mixture-of-Experts (MoE) — ampliou a capacidade de raciocínio sobre programas longos e multiarquivo (SILICONFLOW, 2026). Na frente de integração, os modelos passaram a ser acoplados a ferramentas e ambientes, primeiro via APIs e, posteriormente, via protocolos de interoperabilidade como o Model Context Protocol (ANTHROPIC, 2025; MODEL CONTEXT PROTOCOL, 2025). Na frente agêntica, a geração deixou de ser um ato único e passou a integrar loops de planejamento, edição, execução e reparo (DONG et al., 2025; XIA et al., 2025).

3.2.2 Aprendizagem de uso de ferramentas

A geração de código efetiva depende da capacidade dos modelos de acionar ferramentas externas — compiladores, gerenciadores de pacotes, sistemas de testes, buscadores — de forma autônoma. O trabalho seminal do Toolformer demonstrou que modelos de linguagem podem aprender, por auto-supervisão, a decidir quando e como chamar APIs externas, ampliando substancialmente suas capacidades (SCHICK et al., 2023). Esse princípio é o fundamento dos agentes modernos: a decisão de uso de ferramentas é internalizada pelo modelo e refinada por feedback dos resultados (YAO et al., 2023; GUO et al., 2025).

3.2.3 Benchmarks funcionais: HumanEval e sucessores

A avaliação da geração de código funcional foi padronizada pelo HumanEval, conjunto de 164 problemas de programação com casos de teste ocultos, cuja métrica $\text{pass}@k$ mede a fração de soluções corretas entre k amostras geradas (CHEN et al., 2021). Essa métrica tornou-se o padrão da indústria e foi sucedida por suítes mais amplas e rigorosas, incluindo MBPL, MultiPL-E e CWEval, que avaliam não apenas a corretude funcional, mas também a segurança das saídas (WANG et al., 2025). O avanço dos resultados nessas suítes — de percentuais de dígito único em 2021 para resultados superiores a 90% em benchmarks funcionais atuais — documenta a rápida maturação da geração de código (LLM-STATS, 2026; BENCHLM.AI, 2026).

3.3 Análise: capacidades e limitações da geração de código

3.3.1 A qualidade funcional é condição necessária, mas insuficiente

A literatura converge na distinção entre corretude funcional e qualidade de engenharia. Estudos quantitativos com 4.442 tarefas Java demonstraram que o código gerado que passa nos testes ainda apresenta probabilidade de 5% a 8% de conter bugs e cerca de 2% de conter vulnerabilidades, sem correlação significativa entre a métrica $\text{pass}@1$ e a qualidade estrutural do código (SABRA; SCHMITT; SONAR, 2025). Resultados análogos foram obtidos na análise multi-linguagem e multi-modelo (arXiv:2502.01853, 2025). A corretude funcional, portanto, não é suficiente para garantir a manutenibilidade ou a segurança dos artefatos produzidos.

3.3.2 Segurança do código gerado

A dimensão de segurança concentra as evidências mais preocupantes. O CyberSecEval 3, da Meta, documentou que modelos líderes produzem código inseguro em 35% a 40% das amostras (BHATTACHALI et al., 2024). O dataset SeCodePLT encontrou vulnerabilidades em 40% a 65% das amostras de código gerado (SCHERMANN et al., 2024). As falhas mais recorrentes incluem ausência de validação de entrada (CWE-20), injeção de SQL (CWE-89), injeção de comandos

(CWE-78), credenciais hardcoded, path traversal e dependências alucinadas (SNYK, 2025; ENDOR LABS, 2025). Estudos específicos em desenvolvimento web confirmam a ocorrência desses padrões em linguagens como PHP (VAVEKANAND et al., 2024).

3.3.3 O paradoxo da confiança

Um dos achados mais contraintuitivos da literatura é o paradoxo da confiança: desenvolvedores que utilizam assistentes de IA escrevem código menos seguro ao mesmo tempo em que acreditam mais na segurança do que produzem (PERRY et al., 2022). O estudo de Stanford que documentou o fenômeno mostrou que os usuários do assistente geravam soluções com maior taxa de vulnerabilidades e, paradoxalmente, expressavam maior confiança na correção de suas respostas (PERRY et al., 2022; SNYK, 2025). Complementarmente, experimentos mostraram que rodadas iterativas de refinamento com IA podem elevar a quantidade de vulnerabilidades críticas em cerca de 38% quando aplicadas a código originalmente seguro (SNYK, 2025).

3.3.4 Manutenibilidade e dívida técnica

A análise da manutenibilidade do código gerado revela padrões estruturais distintos. O estudo de ZHU, TSANTALIS e RIGBY (2026) identificou uma “assinatura de máquina” na dívida técnica de código produzido por LLMs e agentes: prevalência de bloat procedural, God Classes e acoplamento cíclico, com a chamada “Lei Inversa Volume-Qualidade”, segundo a qual o volume de código gerado é preditor de degradação estrutural. O estudo demonstrou ainda que nem a correteza funcional nem prompts detalhados evitam essa degradação (ZHU; TSANTALIS; RIGBY, 2026). Em contraste, o estudo de SANTA MOLISON et al. (2025) encontrou indicadores mistos de manutenibilidade e confiabilidade comparados ao código humano, sublinhando a heterogeneidade dos resultados segundo tarefa e modelo.

3.3.5 Heterogeneidade dos resultados

Os resultados da geração de código variam fortemente segundo a natureza da tarefa. Tarefas autocontidas de programação competitiva são resolvidas com alta taxa de sucesso, enquanto tarefas de integração, refatoração em bases legadas e manutenção de sistemas de grande porte apresentam desempenho substancialmente inferior (GUO et al., 2025; JIANG; LO; LIU, 2025). A literatura explica essa heterogeneidade pela necessidade de contexto de projeto — histórico, convenções, arquitetura existente — que os benchmarks funcionais não capturam (BENCHMARKING AGENTS, 2026).

3.4 Síntese Parcial

Os modelos de linguagem evoluíram de geradores de trechos para sistemas capazes de produzir soluções completas e, posteriormente, para componentes de agentes autônomos (CHEN et al.,

2021; DONG et al., 2025). A avaliação funcional avançou do HumanEval para suítes que medem também segurança e qualidade (WANG et al., 2025). As evidências, contudo, são inequívocas quanto à insuficiência da corretude funcional: o código gerado apresenta riscos de segurança sistemáticos (BHATTACHALI et al., 2024; SCHERMANN et al., 2024), paradoxo de confiança nos usuários (PERRY et al., 2022) e padrões estruturais de dívida técnica (ZHU; TSANTALIS; RIGBY, 2026). Esses achados fundamentam a necessidade de camadas de verificação e governança que serão examinadas nas seções 7 e 8, e delimitam o que os agentes de software — objeto da seção 4 — podem realisticamente entregar.



4 AGENTES DE SOFTWARE AUTÔNOMOS

4.1 Contextualização

A transição da geração pontual de código para a execução autônoma de tarefas completas de engenharia materializa-se nos agentes de software. Um agente de software é um sistema baseado em LLM que planeja, navega em repositórios, edita arquivos, executa comandos e valida resultados de forma iterativa, com o objetivo de resolver tarefas reais de engenharia de software (YANG et al., 2024; LIU et al., 2024). Esta seção examina a arquitetura desses sistemas — interfaces, loops de raciocínio, memória e ferramentas — e os sistemas de referência que estabeleceram o estado da arte: SWE-agent, OpenHands e Devin.

4.2 Referencial Teórico: arquitetura dos agentes

4.2.1 A Interface Agente-Computador (ACI)

O conceito central introduzido pelo SWE-agent é o de *Agent-Computer Interface* (ACI): o conjunto de primitivas de observação e ação pelas quais um modelo de linguagem interage com um computador (YANG et al., 2024). A pesquisa demonstrou que o desenho da ACI — observações concisas, comandos de alto nível, feedback estruturado de testes — tem impacto no desempenho do agente maior do que a escolha do modelo subjacente. O SWE-agent obteve, com ACI bem projetada, desempenho superior ao de modelos maiores usados sem tais interfaces, em uma época em que os benchmarks agênticos estavam sendo estabelecidos (YANG et al., 2024; CODESOTA, 2026).

4.2.2 Loops de raciocínio-ação

A operação dos agentes baseia-se em loops iterativos de raciocínio e ação. O framework ReAct estabeleceu o padrão: intercalar raciocínio explícito (thoughts) e ações (acts), permitindo que o modelo planeje, execute e observe os efeitos de suas decisões (YAO et al., 2023). O Reflexion estendeu o padrão com um mecanismo de memória verbal: o agente registra reflexões sobre suas falhas em uma memória episódica e as utiliza em tentativas subsequentes, configurando uma forma de aprendizado por reforço verbal (SHINN et al., 2023). Esses mecanismos compõem a base cognitiva dos agentes de engenharia atuais (DONG et al., 2025; JIANG; LO; LIU, 2025).

4.2.3 Memória e ferramentas

A autonomia dos agentes depende de duas infraestruturas complementares: memória e ferramentas. A memória organiza o contexto — histórico de decisões, estado do repositório, resultados de testes — em camadas de curto e longo prazo, permitindo ao agente manter coerência em tarefas longas (LIU et al., 2024; JIN et al., 2024). O uso de ferramentas, por sua vez, expande o escopo de ação: compiladores, linters, gerenciadores de testes e navegadores são acionados pelo agente como primitivas de ação, seguindo o princípio do Toolformer de aprendizagem autônoma de uso de ferramentas (SCHICK et al., 2023). A integração padronizada dessas ferramentas é objeto do Model Context Protocol, que define primitivas de recursos, prompts, ferramentas e amostragem (MODEL CONTEXT PROTOCOL, 2025; NASCIMENTO et al., 2025).

4.3 Análise: sistemas de referência

4.3.1 SWE-agent

O SWE-agent, desenvolvido na Universidade de Princeton, é a referência acadêmica do campo: estabeleceu as ACIs, o padrão de avaliação e a arquitetura de resolução de issues baseada em execução local (YANG et al., 2024). Sua abordagem de interface mínima com observações enxutas demonstrou ganhos significativos de desempenho em SWE-bench em relação a sistemas anteriores, e seu código foi mantido posteriormente na forma do mini-swe-agent (AICOLIES, 2026). Sua importância é metodológica: definiu as condições de experimentação controlada em tarefas reais de engenharia.

4.3.2 OpenHands

O OpenHands (ex-OpenDevin) é a plataforma open-source mais difundida para agentes de desenvolvimento, com runtime em sandbox Docker/Kubernetes, suporte a agentes paralelos e agnosticismo de modelo (WANG et al., 2024; AICOLIES, 2026). Sua adoção por empresas como Apple, Google, Amazon, Netflix e NVIDIA demonstra a convergência entre pesquisa acadêmica e prática industrial. A plataforma destaca-se pela segurança de execução em ambiente isolado e pela reprodutibilidade dos experimentos (WANG et al., 2024).

4.3.3 Devin e os agentes gerenciados em nuvem

O Devin, da Cognition, representa a categoria dos agentes gerenciados em nuvem: um “engenheiro de software IA” com editor, navegador e terminal próprios, integrações com GitHub, Slack, Jira e Linear, e execução assíncrona de tarefas (AICOLIES, 2026; TOOLBOXKART, 2026). Essa arquitetura desloca a execução do ambiente local do desenvolvedor para infraestrutura remota, com implicações de governança, custo e observabilidade. O OpenAI Codex,

relançado em 2025, segue o mesmo modelo de execução assíncrona em sandbox em nuvem, com criação autônoma de pull requests (TOOLBOXKART, 2026).

4.3.4 Abordagens alternativas e o debate da complexidade

A literatura registra um debate relevante sobre a necessidade de agentes complexos. O sistema Agentless demonstrou que uma pipeline simples de localização e edição, sem raciocínio agêntico iterativo, alcança desempenho competitivo em SWE-bench (XIA et al., 2025). Esse resultado contesta a premissa de que autonomia plena é necessária para resolver issues reais e sugere que a arquitetura de pipeline — contexto bem montado, localização precisa e edição cirúrgica — pode ser mais determinante que a sofisticação do loop de raciocínio (XIA et al., 2025). Complementarmente, estudos de context engineering mostram que a qualidade do contexto fornecido ao modelo supera o prompt engineering na resolução de tarefas (MURUGESAN, 2026).

4.4 Síntese Parcial

Os agentes de software autônomos representam a materialização operacional do ADD. Sua arquitetura assenta-se em ACIs bem desenhadas (YANG et al., 2024), loops de raciocínio-ação com reflexão (YAO et al., 2023; SHINN et al., 2023), memória estruturada e uso de ferramentas (SCHICK et al., 2023; LIU et al., 2024). Os sistemas de referência — SWE-agent, OpenHands e Devin — definem três modelos de operação: acadêmico, plataforma open-source e serviço gerenciado em nuvem (AICOLIES, 2026). O debate entre autonomia plena e pipelines simplificados (XIA et al., 2025) qualifica a interpretação dos resultados de benchmark, que será aprofundada na seção 7. A existência de múltiplos agentes interagindo entre si e com ferramentas padronizadas levanta, por sua vez, as questões de orquestração e interoperabilidade examinadas na seção 5.



5 SISTEMAS MULTIAGENTES E PROTOCOLOS DE INTEROPERABILIDADE

5.1 Do agente isolado à colaboração entre agentes

A trajetória da engenharia de software dirigida por inteligência artificial (IA) iniciou-se com agentes individuais capazes de planejar, navegar em repositórios, editar arquivos, executar comandos e validar resultados de forma iterativa (YANG et al., 2024). À medida que esses sistemas amadureceram, contudo, tornou-se evidente que um único agente, por mais capaz que seja, opera com horizontes limitados: seu contexto é finito, sua especialidade é restrita e sua capacidade de verificação é parcial. A resposta natural a essa limitação foi a coordenação — múltiplos agentes colaborando sob diferentes papéis, trocando mensagens e compartilhando artefatos (HONG et al., 2024; DONG et al., 2025).

A distinção entre automação monolítica e colaboração multiagente não é meramente quantitativa. Um agente único executa um pipeline linear de percepção-planejamento-ação; um sistema multiagente introduz concorrência de perspectivas, revisão cruzada e especialização funcional. Essa diferença estrutural aproxima o processo de produção de software do modelo organizacional humano, no qual papéis como arquiteto, implementador, testador e revisor operam em paralelo e se interdependem (HONG et al., 2024). A literatura sobre agentes LLM mostra que a decomposição de tarefas complexas em subagentes com responsabilidades bem definidas tende a melhorar a aderência às instruções e a qualidade dos artefatos produzidos, ao custo de maior sobrecarga de comunicação e de orquestração (DONG et al., 2025; GUO et al., 2025).

5.2 Arquiteturas de orquestração: o caso MetaGPT

Entre as arquiteturas multiagentes propostas, o MetaGPT é a referência mais citada. O sistema decompõe a produção de software na sequência clássica de fases — análise de requisitos, design, implementação, teste e documentação — e atribui cada fase a um agente especializado: product manager, arquiteto, engenheiro de software e gerente de projeto (HONG et al., 2024). O diferencial do MetaGPT reside na comunicação estruturada: em vez de trocar mensagens de texto livre, os agentes publicam artefatos intermediários em uma “linha de montagem” de documentos, incluindo requisitos, diagramas de arquitetura e especificações de interfaces. A codificação de procedimentos organizacionais em templates — o chamado “meta-programming” — reduz a ambiguidade das mensagens e confere aos agentes um repositório compartilhado de conhecimento de domínio (HONG et al., 2024).

Os resultados reportados indicam que, em tarefas de benchmark, a abordagem multiagente do MetaGPT supera pipelines de geração única de código, sobretudo quando a tarefa exige múltiplas etapas de refinamento (HONG et al., 2024). É importante, porém, situar esses resultados: os ganhos concentram-se em problemas de escopo médio, nos quais a decomposição em subtarefas é viável e mensurável. Em tarefas abertas de engenharia de software real, a vantagem comparativa dos sistemas multiagentes permanece tema de debate, pois a orquestração adiciona latência e custo de tokens que precisam ser compensados por ganhos efetivos de qualidade (XIA et al., 2025; GUO et al., 2025).

5.3 Ciclos de reflexão e autocrítica entre agentes

A colaboração entre agentes não se limita à divisão de tarefas; envolve também mecanismos de autoavaliação e correção. O framework ReAct estabeleceu a base ao intercalar raciocínio e ação, permitindo que o agente observe os resultados de suas ações e revise seu plano (YAO et al., 2023). O Reflexion amplia esse princípio ao introduzir “memória verbal”: após uma tentativa, o agente gera uma avaliação textual do próprio desempenho e a utiliza como insumo para a tentativa seguinte (SHINN et al., 2023). Em sistemas multiagentes, essa capacidade individual de reflexão é escalada para a revisão cruzada — um agente implementa, outro testa e um terceiro audita o resultado, em ciclo que se repete até o atendimento dos critérios de aceitação (HONG et al., 2024; DONG et al., 2025).

Há evidências de que ciclos de refinamento iterativo melhoram a correteude funcional, mas não a segurança. Estudo empírico demonstrou que cinco rodadas de refinamento com IA elevaram a proporção de vulnerabilidades críticas em aproximadamente 38% sobre código inicialmente seguro (Snyk, 2025). Esse resultado tem implicação direta para a arquitetura multiagente: a revisão cruzada melhora a conformidade com requisitos explícitos, porém tende a reforçar padrões de raciocínio já estabelecidos pelo modelo, incluindo falhas de segurança conhecidas. A validação, portanto, precisa ser assistida por ferramentas externas — analisadores estáticos, testes de segurança e revisão humana — e não apenas por outros agentes (PERRY et al., 2022; Snyk, 2025).

5.4 Protocolos de interoperabilidade entre agentes

A proliferação de agentes e de ferramentas criou um problema de integração: cada agente precisava de adaptadores proprietários para cada ferramenta, e a comunicação entre agentes de fornecedores distintos exigia acordos ad hoc. A padronização avançou por meio de protocolos abertos, analisados em revisão sistemática da literatura (NASCIMENTO et al., 2025):

O Model Context Protocol (MCP), proposto pela Anthropic, ataca a integração agente-ferramenta. O protocolo define um modelo cliente-servidor no qual o agente (cliente) descobre capacidades e invoca ferramentas expostas por servidores, com transporte padronizado e comunicação JSON-RPC. A adoção do MCP generalizou-se rapidamente: ambientes como Claude

Code o utilizam nativamente, e o ecossistema de servidores MCP passou a incluir bancos de dados, sistemas de arquivos, navegadores e APIs corporativas (ANTHROPIC, 2025; NASCIMENTO et al., 2025).

O Agent Communication Protocol (ACP) ataca o problema complementar: a comunicação entre agentes, definindo mensagens estruturadas de diálogo, descoberta de capacidades e encerramento de sessões. O Agent-to-Agent Protocol (A2A) propõe interoperabilidade ponto a ponto com base em “cards de agente” — descrições declarativas de habilidades que permitem que um agente descubra e delegue tarefas a outro. O Agent Network Protocol (ANP) estende o modelo para redes distribuídas de agentes, com roteamento de mensagens entre nós heterogêneos (NASCIMENTO et al., 2025).

A coexistência desses protocolos reflete uma divisão natural de fronteiras: MCP para ferramentas, ACP para diálogo entre agentes, A2A para delegação e descoberta, e ANP para redes. A consolidação é incipiente, e a interoperabilidade plena — na qual um agente de um fornecedor delega tarefas a um agente de outro fornecedor com garantias de contrato — permanece objetivo de pesquisa mais do que realidade operacional (NASCIMENTO et al., 2025; TEQNOVOS, 2025).

5.5 Padrões emergentes e implicações para as equipes de desenvolvimento

A adoção corporativa de sistemas multiagentes já produziu padrões observáveis. A Stripe opera os “Minions”, agentes autônomos em nuvem que executam tarefas de código com mínima intervenção humana; Amazon adota política de revisão obrigatória por engenheiro sênior para qualquer código gerado por IA; e Google e Microsoft reportam que cerca de 30% do código novo já é escrito ou assistido por IA (ALURA, 2026). Esses casos ilustram um espectro de governança: da autonomia supervisionada à revisão humana obrigatória, o grau de independência conferido aos agentes é uma decisão organizacional, não meramente técnica (STRAY et al., 2025; CUI et al., 2025).

No plano técnico, dois padrões se destacam. O primeiro é a adoção de plataformas abertas e extensíveis, como o OpenHands, que permitem integrar modelos, ferramentas e agentes customizados sob uma interface comum de execução (WANG et al., 2024). O segundo é o design de fluxos de trabalho “agentless” — pipelines determinísticos de planejamento e reparo que alcançam desempenho competitivo sem a complexidade de orquestração de agentes autônomos (XIA et al., 2025). A existência desse contraste reforça a lição central desta seção: a colaboração entre agentes deve ser introduzida quando a decomposição da tarefa e a especialização de papéis produzirem ganhos mensuráveis, e não como fim em si (DONG et al., 2025; GUO et al., 2025).

As implicações para equipes humanas são profundas. O engenheiro de software deixa de ser o executor único de tarefas e passa a atuar como especificador de fluxos de agentes, avaliador de artefatos e gestor de exceções (JIANG et al., 2025). A competência crítica desloca-se da escrita de código para a orquestração: saber quando delegar, como validar e onde intervir. Ao mesmo tempo, a rastreabilidade do processo exige instrumentação: logs de decisão, versões

de prompts, trilhas de ferramentas invocadas e métricas de custo por tarefa (STRAY et al., 2025; CUI et al., 2025). Sistemas multiagentes, portanto, não eliminam a necessidade de controle humano; redefinem sua posição no ciclo de produção (ALURA, 2026; TOOLBOXKART, 2026).

5.6 Síntese parcial

esta seção examinou a evolução dos agentes isolados para sistemas multiagentes e os protocolos que viabilizam sua interoperabilidade. Verificou-se que (i) a colaboração entre agentes especializados supera pipelines monolíticos em tarefas decomponíveis, com o MetaGPT como arquétipo de orquestração por artefatos estruturados; (ii) ciclos de reflexão e revisão cruzada melhoram a corretude funcional, mas não a segurança, exigindo validação por ferramentas externas e revisão humana; (iii) os protocolos MCP, ACP, A2A e ANP organizam a integração agente-ferramenta e agente-agente em camadas complementares ainda em consolidação; e (iv) a adoção corporativa vem produzindo padrões de governança e plataformas abertas que redefinem o papel do engenheiro de software como orquestrador e avaliador de agentes. Esses achados fornecem a base para o exame do ecossistema de ferramentas e práticas de mercado, tema da próxima seção (NASCIMENTO et al., 2025; HONG et al., 2024; XIA et al., 2025).



6 ECOSSISTEMA DE FERRAMENTAS E PRÁTICAS DE MERCADO

6.1 Panorama histórico e consolidação do mercado

A oferta de ferramentas de IA para engenharia de software evoluiu em duas ondas. A primeira, inaugurada pelo GitHub Copilot em 2021, popularizou o paradigma do autocompletar: um modelo de linguagem (originalmente o Codex, da OpenAI) sugeria continuções de código no editor, integrado diretamente ao fluxo de trabalho do desenvolvedor (TOOLBOXKART, 2026). A segunda onda, a partir de 2024 e 2025, deslocou o foco do preenchimento de código para a execução autônoma de tarefas: agentes que planejam, editam arquivos, executam comandos e abrem pull requests sem intervenção contínua (BENCHLM.AI, 2026; AICOOOLIES, 2026).

Essa transição não foi apenas técnica, mas econômica. O modelo de negócio migrou de licenças individuais de assistentes para plataformas empresariais com governança centralizada — audit logs, políticas de uso, controle de modelos e métricas de adoção (TOOLBOXKART, 2026). A oferta atual concentra-se em três categorias: assistentes de IDE (Copilot), agentes em nuvem (OpenAI Codex) e agentes locais/síncronos (Claude Code), cada uma com trade-offs próprios de latência, privacidade e autonomia (TOOLBOXKART, 2026; OFLIGHT, 2026).

6.2 Assistentes de IDE: a camada ubíqua

O GitHub Copilot permanece a referência da primeira categoria, sustentado pela profundidade de integração no editor e pela gestão empresarial: desde fevereiro de 2026 passou a permitir a seleção de modelos (GPT-5.4, GPT-5.3-Codex, Claude Opus 4.6, Claude Sonnet 4.6 e Gemini 2.0 Pro), com preço aproximado de dez dólares mensais (TOOLBOXKART, 2026). A novidade estrutural é a neutralidade de modelos: o assistente deixou de ser amarrado a um único fornecedor, transformando-se em um intermediário que compete por integração e governança, não por modelo proprietário (TOOLBOXKART, 2026; OFLIGHT, 2026).

O valor dessa categoria reside na ubiquidade e no baixo atrito: as sugestões aparecem no ponto exato da edição, sem mudança de contexto. Evidências experimentais em larga escala sustentam o ganho de produtividade: estudo randomizado com 4.867 desenvolvedores de Microsoft, Accenture e empresas Fortune 100 registrou aumento de 26,08% nas tarefas concluídas por semana com o uso do Copilot (medido por pull requests, commits e builds), com ganhos maiores entre desenvolvedores menos experientes (CUI et al., 2025). Estudo posterior da própria Microsoft corroborou o efeito (INFOQ, 2024). As limitações também são documentadas: a aceitação acrítica de sugestões pode produzir código correto funcionalmente, porém inseguro ou de baixa manutenibilidade (Snyk, 2025; ENDOR LABS, 2025).

6.3 Agentes em nuvem: execução assíncrona e paralelismo

A segunda categoria é exemplificada pelo novo OpenAI Codex, lançado em maio de 2025: um agente de codificação autônomo que executa em sandbox em nuvem, processando tarefas de forma assíncrona e paralela, criando pull requests e integrando-se a repositórios remotos (TOOLBOXKART, 2026). O desempenho em SWE-bench Verified situa-se entre 78% e 85% com o modelo GPT-5.3-Codex (BENCHLM.AI, 2026). O atributo distintivo dessa arquitetura é a delegação: o desenvolvedor especifica a tarefa e recebe um pull request, aproximando o fluxo da revisão de código tradicional, sem a necessidade de acompanhar o raciocínio do agente em tempo real (TOOLBOXKART, 2026; AICOOLIES, 2026).

O paralelismo é o segundo atributo: múltiplas tarefas podem ser executadas simultaneamente em sandboxes isolados, o que reescala o throughput individual — um único desenvolvedor pode despachar dezenas de mudanças em paralelo (TOOLBOXKART, 2026). Essa capacidade reconfigura a engenharia de software como prática de orquestração de trabalho delegado, com consequências para o dimensionamento de equipes e para o papel da revisão (ALURA, 2026; CUI et al., 2025).

6.4 Agentes locais e síncronos: controle e contexto profundo

A terceira categoria, representada pelo Claude Code, opera no terminal do desenvolvedor, de forma síncrona e com acesso local ao repositório, janela de contexto profunda (até um milhão de tokens) e suporte nativo ao Model Context Protocol (MCP) para acoplamento a ferramentas (TOOLBOXKART, 2026). Lidera o SWE-bench Verified com cerca de 80,8% (Claude Opus 4.6) (BENCHLM.AI, 2026). O trade-off dessa arquitetura é a privacidade: o código permanece no ambiente corporativo, sem transmissão para sandboxes de terceiros, o que atende requisitos de segurança e conformidade de setores regulados (TOOLBOXKART, 2026; NASCIMENTO et al., 2025).

Comparações diretas entre as três categorias mostram que não há ferramenta dominante: assistentes maximizam a fluidez da edição; agentes em nuvem maximizam o paralelismo e a delegação; agentes locais maximizam o controle e a confidencialidade (TOOLBOXKART, 2026; AICOOLIES, 2026). Organizações maduras combinam as categorias segundo o tipo de tarefa — autocompletar para mudanças triviais, agente em nuvem para refatorações bem especificadas e agente local para código sensível — em vez de adotar uma única ferramenta (TOOLBOXKART, 2026; ALURA, 2026).

6.5 Práticas de adoção: do piloto à governança

A literatura e os relatos corporativos convergem em um roteiro de adoção. A fase inicial é o piloto controlado, com métricas de produtividade e qualidade definidas a priori, como no estudo de caso da NAV IT, que ampliou seu time de cem para duzentos e cinquenta desenvolvedores com Copilot entre 2023 e 2025 — estudo longitudinal que documentou adoção heterogênea,

ceticismo de engenheiros seniores e custo de verificação das sugestões (STRAY et al., 2025; NAV IT, 2025). A segunda fase é a definição de políticas de uso: quais tarefas podem ser delegadas a agentes, quais exigem revisão humana obrigatória e quais estão proibidas (ALURA, 2026).

A terceira fase é a instrumentação: telemetria de uso, taxas de aceitação, latência de revisão, densidade de defeitos introduzidos e custo por tarefa (CUI et al., 2025; STRAY et al., 2025). Sem instrumentação, a adoção de ferramentas de IA permanece refém de percepções subjetivas — fenômeno já documentado como “paradoxo da confiança”, em que desenvolvedores que usam assistentes consideram seu código mais seguro justamente quando ele apresenta mais vulnerabilidades (PERRY et al., 2022; Snyk, 2025). A governança efetiva, portanto, não restringe o uso; torna-o mensurável e auditável (ENDOR LABS, 2025; TOOLBOXKART, 2026).

6.6 Síntese parcial

Esta seção caracterizou o ecossistema de ferramentas em três categorias — assistentes de IDE, agentes em nuvem e agentes locais — e as práticas de mercado associadas. Verificou-se que (i) o mercado consolidou-se em duas ondas: autocompletar e execução autônoma de tarefas; (ii) cada categoria apresenta trade-offs distintos de fluidez, paralelismo, privacidade e governança; (iii) a evidência empírica de ganho de produtividade é robusta, com destaque para o aumento de 26,08% documentado em estudo randomizado (CUI et al., 2025); e (iv) a adoção madura combina piloto controlado, políticas de uso e instrumentação contínua, contornando os vieses de percepção documentados na literatura (PERRY et al., 2022; STRAY et al., 2025). Esses elementos preparam o exame da avaliação de agentes por benchmarks, tema da próxima seção (TOOLBOXKART, 2026; BENCHLM.AI, 2026).



7 AVALIAÇÃO DE AGENTES: BENCHMARKS E LACUNAS DE MEDIÇÃO

7.1 A necessidade de avaliação padronizada

A avaliação de agentes de IA para engenharia de software exige instrumentos padronizados capazes de comparar modelos, arquiteturas e ferramentas sob condições controladas. Sem benchmarks confiáveis, a seleção de ferramentas apoia-se em percepções subjetivas, marketing de fornecedores e avaliações anedóticas — situação incompatível com a decisão técnica informada exigida pela engenharia de software (JIMENEZ et al., 2024; GUO et al., 2025). A literatura registra esforço intenso de construção de benchmarks desde a primeira onda de modelos geradores de código, com progressiva sofisticação metodológica (DONG et al., 2025; GUO et al., 2025).

A evolução dos benchmarks acompanha a evolução dos próprios sistemas avaliados: enquanto a geração autônoma de funções isoladas exigia conjuntos de problemas de programação competitiva, os agentes modernos, que operam sobre repositórios reais, demandam tarefas integrais de engenharia — ler um repositório, localizar o ponto de alteração, implementar a mudança e validá-la com os testes existentes (JIMENEZ et al., 2024; YANG et al., 2024). Essa transição reposicionou o objeto da medição: não se avalia mais apenas a corretude sintática do código produzido, mas a capacidade de resolver problemas reais de manutenção de software (JIMENEZ et al., 2024).

7.2 O SWE-bench e o SWE-bench Verified

O SWE-bench consolidou-se como referência central da área. O benchmark é construído a partir de 2.294 problemas reais extraídos de 12 repositórios Python de código aberto (django, sympy, scikit-learn, matplotlib e outros), cada problema consistindo em um issue do GitHub, um patch de referência gerado por desenvolvedores humanos e um conjunto de testes que validam a solução (JIMENEZ et al., 2024). A tarefa do agente é gerar o patch que resolve o issue; o agente é considerado bem-sucedido se os testes previamente falhando passam e os demais continuam passando (JIMENEZ et al., 2024).

O SWE-bench Verified é um subconjunto de 500 problemas validados manualmente por humanos, criado para eliminar ambiguidades e casos com descrições insuficientes que contaminavam a métrica original (JIMENEZ et al., 2024). Essa curadoria elevou a confiabilidade das comparações e tornou-se o número mais citado nos relatórios de fornecedores: o Codex alcança

entre 78% e 85% (BENCHLM.AI, 2026; TOOLBOXKART, 2026), e o Claude Code lidera com aproximadamente 80,8% na configuração mais recente (BENCHLM.AI, 2026). A divulgação desses números nos materiais de marketing, entretanto, raramente informa as condições exatas de execução — número de tentativas, modelos, custos e infraestrutura — o que limita a comparabilidade entre fornecedores (BENCHLM.AI, 2026; TOOLBOXKART, 2026).

7.3 Outros benchmarks e a medição de habilidades específicas

Além do SWE-bench, o campo desenvolveu benchmarks para habilidades específicas da engenharia de software. O HumanEval, proposto com o Codex original, avalia a síntese de funções a partir de descrições docstring (CHEN et al., 2021). O CyberSecEval 3 (Meta) mede a propensão de modelos a gerar código inseguro, revelando que modelos líderes produzem código vulnerável em 35% a 40% das tarefas, com falhas recorrentes de validação de entrada (CWE-20), injeção de SQL (CWE-89) e injeção de comandos (CWE-78) (BHATTACHALI et al., 2024; Snyk, 2025). O SeCodePLT, por sua vez, estima que 40% a 65% das amostras de código gerado contêm vulnerabilidades, embora a maioria não acione gatilhos de segurança em execução (SCHERMANN et al., 2024; ENDOR LABS, 2025).

Benchmarks de corretude funcional convivem com métricas de qualidade estrutural. Estudo da SonarQube com 4.442 tarefas Java constatou que mesmo código que passa nos testes funcionais carrega probabilidade de 5% a 8% de conter bugs e cerca de 2% de conter vulnerabilidades (SABRA et al., 2025). A medição de segurança também revela efeitos contraintuitivos: o “paradoxo da confiança” documenta que desenvolvedores que utilizam assistentes de IA consideram seu código mais seguro exatamente quando ele apresenta mais vulnerabilidades (PERRY et al., 2022; Snyk, 2025). Em conjunto, esses resultados demonstram que a corretude funcional é condição necessária, mas insuficiente, para a qualidade do código gerado (SABRA et al., 2025; SCHERMANN et al., 2024).

7.4 Lacunas metodológicas dos benchmarks atuais

A despeito do avanço, os benchmarks apresentam lacunas metodológicas significativas. A primeira é a contaminação: problemas públicos, replicados em múltiplos benchmarks, podem integrar os corpora de treinamento dos modelos, inflando os resultados de forma artificial (JIMENEZ et al., 2024; GUO et al., 2025). A segunda é a limitação de domínio: o SWE-bench concentra-se em Python e em poucos repositórios, restringindo a generalização para outras linguagens, ecossistemas e tipos de tarefa (JIMENEZ et al., 2024). A terceira é a ausência de medição econômica: poucos benchmarks reportam custo por solução (tokens, chamadas, tempo) — variável decisiva para a adoção empresarial (TOOLBOXKART, 2026; CUI et al., 2025).

A quarta lacuna é a não medição de atributos emergentes da prática real: capacidade de diálogo em linguagem natural, gestão de contexto longo, aderência a políticas de repositório e qualidade do código sob revisão humana (STRAY et al., 2025; GUO et al., 2025). Benchmarks

avaliam o desfecho — o patch correto — mas não o processo, e o processo é precisamente onde as organizações investem em governança (ENDOR LABS, 2025; ALURA, 2026). Por fim, há o problema da flutuação temporal: modelos e configurações mudam rapidamente, tornando obsoletas em meses comparações publicadas com rigor estatístico (BENCHLM.AI, 2026).

7.5 Rumos da avaliação: do benchmark à medição organizacional

A resposta às lacunas não é o abandono dos benchmarks, mas sua complementação por medição organizacional contínua. As práticas emergentes combinam: (i) benchmarks públicos para seleção inicial de ferramentas e modelos; (ii) avaliações internas com tarefas representativas do domínio da organização, incluindo linguagens e padrões próprios; (iii) telemetria de produção, com métricas de taxa de aceitação, tempo de revisão, defeitos introduzidos e custo por tarefa (CUI et al., 2025; STRAY et al., 2025); e (iv) auditorias periódicas de segurança do código gerado, por análise estática e testes dinâmicos (Snyk, 2025; ENDOR LABS, 2025).

Estudos de adoção em larga escala fornecem a evidência de que a medição organizacional é viável e informativa: o estudo randomizado com 4.867 desenvolvedores mensurou ganho de 26,08% nas tarefas concluídas por semana, e o estudo longitudinal da NAV IT documentou custo de verificação das sugestões e adoção heterogênea entre engenheiros (CUI et al., 2025; NAV IT, 2025). Esses estudos indicam que a questão central da avaliação deslocou-se de “qual modelo resolve mais issues” para “como o sistema de IA + humano + processos se comporta no contexto específico da organização” (STRAY et al., 2025; GUO et al., 2025).

7.6 Síntese parcial

Esta seção examinou o estado da arte e as lacunas da avaliação de agentes. Ficou evidenciado que (i) o SWE-bench e sua versão Verified tornaram-se a referência de comparação, com curadoria manual de 500 problemas e liderança do Claude Code (80,8%) e do Codex (78-85%); (ii) benchmarks complementares medem segurança (CyberSecEval 3, SeCodePLT) e qualidade estrutural (SonarQube), revelando que código funcionalmente correto pode conter vulnerabilidades; (iii) as lacunas metodológicas incluem contaminação de treino, domínio restrito, ausência de custo e foco exclusivo no desfecho; e (iv) a tendência é a combinação de benchmarks públicos com medição organizacional contínua, alinhada às decisões de governança (JIMENEZ et al., 2024; CUI et al., 2025; STRAY et al., 2025). A próxima seção examina as dimensões de qualidade, segurança e dívida técnica do código gerado (Snyk, 2025; ENDOR LABS, 2025).



8 QUALIDADE, SEGURANÇA E DÍVIDA TÉCNICA DO CÓDIGO GERADO

8.1 Corretude funcional versus qualidade estrutural

A avaliação automatizada de agentes de IA concentrou-se historicamente na corretude funcional — se o código gerado resolve a tarefa proposta, aferida por testes. A evidência acumulada, contudo, demonstra que corretude e qualidade estrutural são dimensões distintas: código que passa nos testes funcionais pode apresentar bugs latentes, vulnerabilidades exploráveis e arquitetura de baixa manutenibilidade (SABRA et al., 2025; SANTA MOLISON et al., 2025). Estudo da SonarQube com 4.442 tarefas Java constatou que mesmo soluções funcionalmente corretas carregam probabilidade de 5% a 8% de conter bugs e cerca de 2% de conter vulnerabilidades (SABRA et al., 2025).

Pesquisa dedicada à manutenibilidade comparou código gerado por LLMs com código escrito por humanos, concluindo que as diferenças mais relevantes situam-se na complexidade ciclomática, no acoplamento entre classes e na legibilidade — atributos que não aparecem em testes funcionais, mas determinam o custo de manutenção ao longo do ciclo de vida (SANTA MOLISON et al., 2025). A implicação prática é direta: processos de adoção de IA que validam exclusivamente por testes automatizados aprovam código com qualidade estrutural inferior, transferindo o custo para a manutenção futura (SANTA MOLISON et al., 2025; GUO et al., 2025).

8.2 Insegurança por padrão: evidências de benchmarks de segurança

A dimensão de segurança concentra as evidências mais robustas de degradação. O CyberSecEval 3, da Meta, demonstrou que modelos líderes produzem código inseguro em 35% a 40% das tarefas, com falhas recorrentes de validação de entrada (CWE-20), injeção de SQL (CWE-89), injeção de comandos (CWE-78), credenciais embutidas no código, path traversal e dependências alucinadas (BHATTACHALI et al., 2024; Snyk, 2025). O SeCodePLT estimou que 40% a 65% das amostras de código gerado contêm vulnerabilidades (SCHERMANN et al., 2024). Avaliação focada em PHP revelou vulnerabilidades e limitações específicas de ecossistemas web menos cobertos pelos corpora de treinamento (VAVEKANAND et al., 2024).

O panorama é agravado pelo “paradoxo da confiança”: desenvolvedores que utilizam assistentes de IA consideram seu código mais seguro precisamente quando ele apresenta mais vulnerabilidades (PERRY et al., 2022; Snyk, 2025). O refinamento iterativo agrava o quadro:

cinco rodadas de refinamento com IA elevaram a proporção de vulnerabilidades críticas em aproximadamente 38% sobre código inicialmente seguro (Snyk, 2025). A interpretação mais aceita é que os ciclos de iteração otimizam a adequação funcional do código ao teste — e não sua postura de segurança, que permanece dependente do padrão estatístico do modelo (Snyk, 2025; SCHERMANN et al., 2024).

8.3 Dívida técnica e a “assinatura de máquina”

A dimensão mais recente da literatura é a dívida técnica estrutural. Análise de código gerado por LLMs em larga escala identificou uma “assinatura de máquina”: o código produzido por agentes acumula padrões de procedural bloat, God Classes, acoplamento excessivo e ausência de camadas de abstração — características estatisticamente distinguíveis do código escrito por humanos (ZHU; TSANTALIS; RIGBY, 2026). A contribuição central do estudo é a chamada Lei Inversa Volume-Qualidade: quanto maior o volume de código gerado por máquinas em um repositório, menor a qualidade estrutural média do conjunto, em contraste com a relação convencional observada em código humano (ZHU; TSANTALIS; RIGBY, 2026).

A metáfora da assinatura é operacionalmente útil: se o código gerado tem características estatísticas identificáveis, ele também pode ser detectado, monitorado e mitigado por ferramentas de análise estática e por políticas de revisão (ZHU; TSANTALIS; RIGBY, 2026; ENDOR LABS, 2025). A dívida técnica, nesse contexto, deixa de ser um conceito difuso e passa a ser mensurável: proporção de código gerado, densidade de padrões de acoplamento, complexidade ciclomática média e débito de refatoração estimado (SANTA MOLISON et al., 2025; ZHU; TSANTALIS; RIGBY, 2026).

8.4 Guardrails organizacionais e técnicos

A resposta prática combina guardrails técnicos e organizacionais. No plano técnico, a literatura e os relatos corporativos convergem para: (i) análise estática obrigatória em todo código gerado, com gates de segurança (SAST) e varredura de dependências (Snyk, 2025; ENDOR LABS, 2025); (ii) testes dinâmicos e de regressão ampliados, incluindo testes de segurança (BHATTACHALI et al., 2024); e (iii) medição contínua de qualidade estrutural, com metas de complexidade e cobertura (SANTA MOLISON et al., 2025; ZHU; TSANTALIS; RIGBY, 2026).

No plano organizacional, os padrões observados incluem revisão humana obrigatória para código gerado, como na política da Amazon que exige revisão de engenheiro sênior antes do merge (ALURA, 2026); limites de autonomia para agentes, diferenciando tarefas delegáveis das que exigem aprovação (STRAY et al., 2025); e instrumentação de produção com métricas de densidade de defeitos introduzidos por origem de código — humano ou gerado (CUI et al., 2025; ENDOR LABS, 2025). A existência desses guardrails não elimina o risco, mas o torna mensurável e gerenciável (Snyk, 2025; ALURA, 2026).

8.5 Síntese parcial

Esta seção examinou as dimensões de qualidade, segurança e dívida técnica do código gerado. Ficou evidenciado que (i) corretude funcional não implica qualidade estrutural, com riscos mensuráveis de bugs latentes e vulnerabilidades; (ii) benchmarks de segurança documentam insegurança por padrão em 35% a 65% dos casos, agravada pelo paradoxo da confiança e pela iteração sem guardrails; (iii) a assinatura de máquina e a Lei Inversa Volume-Qualidade demonstram que código gerado acumula dívida técnica estrutural identificável; e (iv) guardrails técnicos (SAST, testes, medição) e organizacionais (revisão humana, limites de autonomia, telemetria) configuram a resposta prática da indústria (ZHU; TSANTALIS; RIGBY, 2026; Snyk, 2025; ALURA, 2026). A próxima seção examina a produtividade e o impacto organizacional da adoção (CUI et al., 2025; STRAY et al., 2025).



9 PRODUTIVIDADE E IMPACTO ORGANIZACIONAL DA ADOÇÃO DE IA

9.1 Evidência experimental de ganhos de produtividade

A questão central da adoção de IA na engenharia de software é empírica: ferramentas generativas aumentam efetivamente a produtividade? A evidência mais robusta provém de estudos randomizados controlados (RCTs) — o padrão-ouro de inferência causal. Estudo conduzido com 4.867 desenvolvedores da Microsoft, da Accenture e de empresas Fortune 100 randomizou participantes entre uso e não uso do GitHub Copilot, registrando aumento de 26,08% nas tarefas concluídas por semana, medido por pull requests, commits e builds (CUI et al., 2025). O ganho foi heterogêneo: desenvolvedores menos experientes obtiveram os maiores incrementos, enquanto profissionais seniores apresentaram ganhos menores, possivelmente em razão de fluxos de trabalho já otimizados (CUI et al., 2025; INFOQ, 2024).

A robustez do resultado decorre do desenho experimental: a randomização controla variáveis de confusão (perfil do desenvolvedor, complexidade da tarefa, contexto do projeto), permitindo atribuir a diferença à intervenção (CUI et al., 2025). Estudos correlacionais e surveys corporativos corroboram a direção do efeito, embora com menor rigor causal (INFOQ, 2024; TOOLBOXKART, 2026). O levantamento anual da DORA reforça o panorama: cerca de 90% dos desenvolvedores já utilizam IA generativa em algum grau, e a percepção dominante é a de que a IA atua como amplificador de desempenho — acelerando a execução de tarefas conhecidas — sem substituir o julgamento humano (DORA; GOOGLE CLOUD, 2025).

9.2 Onde a produtividade aumenta e onde não muda

A análise fina das evidências sugere que o ganho de produtividade concentra-se em tarefas bem delimitadas: geração de código boilerplate, escrita de testes, documentação, refatoração mecânica e prototipagem (CUI et al., 2025; GUO et al., 2025). Nessas tarefas, o assistente reduz o tempo de digitação e de busca, liberando atenção do desenvolvedor para decisões de design (STRAY et al., 2025). O custo oculto documentado é a verificação: sugestões precisam ser lidas, compreendidas e testadas, e o tempo de verificação pode compensar parte do ganho de escrita — fenômeno observado no estudo longitudinal da NAV IT, que registrou custo de verificação das sugestões e adoção heterogênea entre engenheiros (NAV IT, 2025).

Em tarefas de alta incerteza — arquitetura, integração entre sistemas legados, análise de requisitos ambíguos e incidentes de produção — as evidências de ganho são fracas ou

inexistentes (DORA; GOOGLE CLOUD, 2025; STRAY et al., 2025). A literatura converge para uma leitura qualificada: a IA generativa não é uma solução de produtividade em si, mas um amplificador que potencializa processos já bem estruturados (DORA; GOOGLE CLOUD, 2025). Organizações com pipelines de CI/CD maduros, testes automatizados e revisão disciplinada extraem mais valor da adoção do que organizações sem essas bases (DORA; GOOGLE CLOUD, 2025; CUI et al., 2025).

9.3 Impacto sobre habilidades e dinâmica de equipes

A adoção de IA altera o mapa de habilidades da engenharia de software. A capacidade de especificar, delegar e avaliar trabalho de agentes — competências de orquestração — ganha relevância relativa, enquanto a proficiência em sintaxe e APIs específicas perde peso (JIANG et al., 2025; TEQNOVOS, 2025). O estudo longitudinal da NAV IT documentou ceticismo entre desenvolvedores seniores e adoção entusiástica entre juniores, sugerindo que a tecnologia pode alterar a distribuição de influência técnica nas equipes (NAV IT, 2025). O efeito sobre a trajetória de aprendizado é ambíguo: ferramentas de IA aceleram a resolução de tarefas, mas podem reduzir o contato com o erro e a reflexão que sustentam o aprendizado profundo (STRAY et al., 2025; GUO et al., 2025).

A literatura também aponta mudança nos padrões de colaboração: revisão de código, originalmente centrada em humanos, passa a incluir a revisão de artefatos gerados por agentes — pull requests produzidos por IA, patches sugeridos e testes sintetizados (ALURA, 2026; STRAY et al., 2025). Políticas como a revisão obrigatória por engenheiro sênior na Amazon configuram a resposta organizacional a essa nova modalidade de trabalho (ALURA, 2026). O papel do revisor desloca-se da verificação de conformidade sintática para a validação de intenção, arquitetura e segurança (ZHU; TSANTALIS; RIGBY, 2026; ENDOR LABS, 2025).

9.4 Riscos organizacionais e condições de sucesso

A adoção de IA carrega riscos organizacionais documentados. O primeiro é o risco de segurança: o paradoxo da confiança demonstra que a percepção de segurança aumenta na mesma proporção em que o código gerado acumula vulnerabilidades (PERRY et al., 2022; Snyk, 2025). O segundo é o risco de dívida técnica estrutural: a assinatura de máquina e a Lei Inversa Volume-Qualidade indicam que o acúmulo de código gerado degrada a qualidade estrutural média do repositório (ZHU; TSANTALIS; RIGBY, 2026). O terceiro é o risco de medição: métricas de produtividade por linhas de código ou por commits tornam-se enganosas quando grande parte do código é gerada e apenas revisada por humanos (STRAY et al., 2025; DORA; GOOGLE CLOUD, 2025).

As condições de sucesso emergem das evidências: (i) instrumentação prévia, com métricas definidas antes da adoção (CUI et al., 2025); (ii) guardrails de segurança, com análise estática obrigatória e testes de segurança do código gerado (Snyk, 2025; ENDOR LABS, 2025); (iii) política explícita de revisão e limites de autonomia para agentes (ALURA, 2026; STRAY

et al., 2025); e (iv) programas de capacitação que formem a competência de orquestração de agentes e de avaliação crítica de artefatos gerados (JIANG et al., 2025; TEQNOVOS, 2025). A combinação dessas condições transforma a adoção de IA de experimento individual em decisão organizacional gerenciável (DORA; GOOGLE CLOUD, 2025).

9.5 Síntese parcial

Esta seção examinou a evidência de produtividade e o impacto organizacional da adoção de IA. Ficou evidenciado que (i) RCTs documentam ganho médio de 26,08% nas tarefas concluídas por semana, concentrado em tarefas bem delimitadas e em desenvolvedores menos experientes; (ii) o ganho exige verificação, que consome parte do tempo economizado, e praticamente não ocorre em tarefas de alta incerteza; (iii) a IA reconfigura habilidades e dinâmica de equipes, com a orquestração substituindo parte da proficiência sintática e políticas de revisão obrigatória emergindo como norma; e (iv) o sucesso da adoção depende de instrumentação, guardrails de segurança, políticas de revisão e capacitação — condições que transformam o uso individual em estratégia organizacional (CUI et al., 2025; DORA; GOOGLE CLOUD, 2025; STRAY et al., 2025). A próxima e última seção consolida as conclusões e os desdobramentos futuros da pesquisa (ZHU; TSANTALIS; RIGBY, 2026; Snyk, 2025).



10 CONSIDERAÇÕES FINAIS

10.1 Síntese da pesquisa

Esta pesquisa investigou o AI Driven Development (ADD) como paradigma emergente de engenharia de software no qual modelos de linguagem de grande escala (LLMs) e agentes de IA participam ativamente de todas as fases do ciclo de vida — especificação, arquitetura, codificação, testes, revisão, implantação e manutenção (TEQNOVOS, 2025; GUO et al., 2025). A análise percorreu os fundamentos do paradigma (seção 2), a evolução dos modelos de linguagem e da geração de código (seção 3), os agentes autônomos (seção 4), os sistemas multiagentes e protocolos de interoperabilidade (seção 5), o ecossistema de ferramentas (seção 6), a avaliação por benchmarks (seção 7), a qualidade, segurança e dívida técnica (seção 8) e a produtividade e o impacto organizacional (seção 9).

Os resultados consolidados indicam que o ADD representa uma mudança estrutural, e não incremental, na produção de software: o locus da automação deslocou-se da compilação e dos builds para a própria atividade de escrita de especificações, código, testes e documentação (SAUVOLA et al., 2024; TERRAGNI; ROOP; BLINCOE, 2024). Evidências randomizadas documentam ganho médio de 26,08% nas tarefas concluídas por semana (CUI et al., 2025), e levantamentos setoriais estimam que cerca de 90% dos desenvolvedores já utilizam IA generativa, com cerca de 30% do código novo sendo escrito ou assistido por IA em grandes empresas (DORA; GOOGLE CLOUD, 2025; ALURA, 2026).

10.2 Principais achados

A pesquisa produziu quatro achados principais. Primeiro, a evidência de produtividade é robusta, porém condicionada: o ganho concentra-se em tarefas bem delimitadas e beneficia mais desenvolvedores menos experientes, enquanto tarefas de alta incerteza permanecem imunes à automação (CUI et al., 2025; DORA; GOOGLE CLOUD, 2025). Segundo, a segurança do código gerado é a fragilidade central: benchmarks como CyberSecEval 3 e SeCodePLT documentam código inseguro em 35% a 65% dos casos, agravado pelo paradoxo da confiança e pela iteração sem guardrails (BHATTAHALI et al., 2024; SCHERMANN et al., 2024; PERRY et al., 2022; Snyk, 2025).

Terceiro, a qualidade estrutural do código gerado degrada com o volume: a assinatura de máquina e a Lei Inversa Volume-Qualidade demonstram que o acúmulo de artefatos produzidos por agentes introduz dívida técnica identificável — procedural bloat, God Classes e acoplamento excessivo (ZHU; TSANTALIS; RIGBY, 2026; SANTA MOLISON et al., 2025). Quarto, a

padronização avança por protocolos abertos de interoperabilidade — MCP, ACP, A2A e ANP — que organizam a integração agente-ferramenta e agente-agente em camadas complementares, ainda em consolidação (NASCIMENTO et al., 2025; ANTHROPIC, 2025).

10.3 Implicações práticas

Para a prática da engenharia de software, os resultados implicam recomendações concretas. A adoção de ferramentas de IA deve ser precedida por instrumentação, com métricas de produtividade, qualidade e segurança definidas antes do início (CUI et al., 2025; STRAY et al., 2025). O código gerado exige guardrails obrigatórios: análise estática (SAST), testes de segurança e revisão humana, como adotado pela Amazon (ALURA, 2026; ENDOR LABS, 2025). A autonomia de agentes deve ser escalada gradualmente, com limites explícitos e rastreabilidade de decisões (JIANG et al., 2025; STRAY et al., 2025).

Para as equipes, a capacitação deve deslocar-se da proficiência sintática para a orquestração: especificar tarefas, avaliar artefatos gerados e intervir nos pontos de exceção (JIANG et al., 2025; TEQNOVOS, 2025). Para a seleção de ferramentas, benchmarks públicos como o SWE-bench Verified devem ser complementados por avaliações no domínio da organização e por telemetria de produção, evitando decisões baseadas em percepções subjetivas (JIMENEZ et al., 2024; BENCHLM.AI, 2026; TOOLBOXKART, 2026).

10.4 Limitações da pesquisa

Esta pesquisa apresenta limitações reconhecidas. A primeira é a dependência da literatura disponível até o período de levantamento: o campo evolui rapidamente, e parte dos resultados reportados pode tornar-se obsoleta em poucos meses (BENCHLM.AI, 2026). A segunda é a heterogeneidade metodológica das fontes: a análise combinou ensaios randomizados, estudos longitudinais, relatórios setoriais e materiais de fornecedores, com diferentes níveis de rigor (DORA; GOOGLE CLOUD, 2025; TOOLBOXKART, 2026). A terceira é a concentração de benchmarks em ecossistemas de linguagens específicos, limitando a generalização para outros domínios (JIMENEZ et al., 2024; GUO et al., 2025).

10.5 Desdobramentos futuros

A pesquisa sugere direções de investigação futura. No plano técnico, são promissoras: a consolidação de protocolos de interoperabilidade e seus efeitos sobre a adoção corporativa (NASCIMENTO et al., 2025); o desenvolvimento de benchmarks que incorporem custo, processo e atributos emergentes da prática real (GUO et al., 2025; BENCHLM.AI, 2026); e a mitigação da assinatura de máquina por técnicas de geração orientada à manutenibilidade (ZHU; TSANTALIS; RIGBY, 2026; SANTA MOLISON et al., 2025). No plano organizacional, destacam-se: estudos longitudinais de longo prazo sobre dívida técnica acumulada (ZHU;

TSANTALIS; RIGBY, 2026; NAV IT, 2025); avaliação de programas de capacitação em orquestração de agentes (JIANG et al., 2025; TEQNOVOS, 2025); e a análise de governança da autonomia de agentes em ambientes regulados (ALURA, 2026; ENDOR LABS, 2025).

10.6 Conclusão

O AI Driven Development consolida-se como paradigma operacional da engenharia de software: a evidência de ganho de produtividade é real, mas condicionada a guardrails de qualidade, segurança e governança (CUI et al., 2025; DORA; GOOGLE CLOUD, 2025). A resposta da indústria combina protocolos abertos de interoperabilidade, benchmarks cada vez mais robustos e políticas explícitas de revisão e autonomia (NASCIMENTO et al., 2025; JIMENEZ et al., 2024; ALURA, 2026). A conclusão central desta pesquisa é que o sucesso da adoção de IA na engenharia de software não depende da capacidade dos modelos, mas da capacidade das organizações de orquestrá-los: mensurar, validar, revisar e integrar agentes ao processo produtivo humano (STRAY et al., 2025; JIANG et al., 2025; GUO et al., 2025).



REFERÊNCIAS

AICOOOLIES. OpenHands vs Devin vs SWE-Agent: Autonomous Coding Agent Comparison. 2026. Disponível em: <https://aicoolies.com/comparisons/openhands-vs-devin-vs-swe-agent>. Acesso em: 08 ago. 2026.

ALURA. IA na Engenharia de Software: Guardrails de Qualidade e Estratégias de Adoção. 2026. Disponível em: <https://www.alura.com.br/conteudo/ia-engenharia-software-guardrails-qualidade-estrategias-adocao>. Acesso em: 08 ago. 2026.

ANTHROPIC. Introducing the Model Context Protocol. 2025. Disponível em: <https://www.anthropic.com/news/model-context-protocol>. Acesso em: 08 ago. 2026.

BAYTECH CONSULTING. Unlocking 2026: The Future of AI-Driven Software Development. 2026. Disponível em: <https://www.baytechconsulting.com/blog/unlocking-ai-software-development-2026>. Acesso em: 08 ago. 2026.

BENCHLM.AI. SWE-bench Verified Benchmark 2026: 44 LLM Scores. 2026. Disponível em: <https://benchlm.ai/benchmarks/sweVerified>. Acesso em: 08 ago. 2026.

BENCHMARKING AGENTS. AI Agent Benchmarks: SWE-bench, WebArena, Agent-Bench, Terminal-Bench, OSWorld, Tau-Bench. 2026. Disponível em: <https://benchmarkingagents.com/agent-benchmarks>. Acesso em: 08 ago. 2026.

BHATTACHALI, Sandeep Kumar et al. CyberSecEval 3: Advancing the Evaluation of Cybersecurity Risks and Capabilities in Large Language Models. Meta, 2024. Disponível em: <https://arxiv.org/abs/2408.01605>. Acesso em: 08 ago. 2026.

CHEN, Mark et al. Evaluating Large Language Models Trained on Code. 2021. Disponível em: <https://arxiv.org/abs/2107.03374>. Acesso em: 08 ago. 2026.

CODESOTA. SWE-bench 2026: Compare Devin, Codex, Claude Code, Cursor, OpenHands, Aider. 2026. Disponível em: <https://www.codesota.com/tasks/swe-bench>. Acesso em: 08 ago. 2026.

CUI, Kevin Zheyuan et al. The Effects of Generative AI on High-Skilled Work: Evidence from Three Field Experiments with Software Developers. SSRN 4945566, 2025. Disponível em: https://economics.mit.edu/sites/default/files/inline-files/draft_copilot_experiments.pdf. Acesso em: 08 ago. 2026.

DONG, Yihong et al. A Survey on Code Generation with LLM-based Agents. 2025. Disponível em: <https://arxiv.org/abs/2508.00083>. Acesso em: 08 ago. 2026.

DORA; GOOGLE CLOUD. State of Cloud-Native Development Report. 2025. Disponível em: <https://dora.dev/reports/>. Acesso em: 08 ago. 2026.

DORA/GOOGLE CLOUD. 2025 State of AI-Assisted Software Development Report. 2025. Disponível em: <https://dora.dev/dora-report-2025/>. Acesso em: 08 ago. 2026.

ENDOR LABS. The Most Common Security Vulnerabilities in AI-Generated Code. 2025. Disponível em: <https://www.endorlabs.com/learn/the-most-common-security-vulnerabilities-in-ai-generated-code>. Acesso em: 08 ago. 2026.

FORRESTER. Predictions 2026: Software Development Goes From Jamming To A Full Orchestra. 2026. Disponível em: <https://www.forrester.com/blogs/predictions-2026-software-development-goes-from-jamming-to-full-orchestra>. Acesso em: 08 ago. 2026.

FUTUREWARNS. AI in Software Development: 2026. 2026. Disponível em: <https://futurewarns.com/ai-in-software-development-2026>. Acesso em: 08 ago. 2026.

GUO, Jiale et al. A Comprehensive Survey on Benchmarks and Solutions in Software Engineering of LLM-Empowered Agentic System. 2025. Disponível em: <https://arxiv.org/abs/2510.09721>. Acesso em: 08 ago. 2026.

HONG, Sirui et al. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. ICLR, 2024. Disponível em: <https://arxiv.org/abs/2308.00352>. Acesso em: 08 ago. 2026.

INFOQ. Study Shows AI Coding Assistant Improves Developer Productivity. 2024. Disponível em: <https://www.infoq.com/news/2024/09/copilot-developer-productivity>. Acesso em: 08 ago. 2026.

INFOQ. AI Is Amplifying Software Engineering Performance, Says the DORA Report. 2026. Disponível em: <https://www.infoq.com/news/2026/03/ai-dora-report/>. Acesso em: 08 ago. 2026.

JIANG, Zhonghao; LO, David; LIU, Zhongxin. Agentic Software Issue Resolution with Large Language Models: A Survey. 2025. Disponível em: <https://arxiv.org/abs/2507.03126>. Acesso em: 08 ago. 2026.

LLM-STATS. SWE-Bench Verified Leaderboard. 2026. Disponível em: <https://llm-stats.com/benchmarks/swe-bench-verified>. Acesso em: 08 ago. 2026.

MINDSHARE. Claude Code vs OpenAI Codex: Which AI Coding Agent Is Better? 2026. Disponível em: <https://www.mindstudio.ai/blog/claude-code-vs-openai-codex-comparison>. Acesso em: 08 ago. 2026.

MODEL CONTEXT PROTOCOL. Specification 2025-11-25. 2025. Disponível em: <https://modelcontextprotocol.io/specification/2025-11-25>. Acesso em: 08 ago. 2026.

MURUGESAN, Thirunaavukkarasu. Enhancing SWE Bench with Context Engineering: A Comparative Study Against Prompt Engineering in LLM-Based Software Tasks. Journal of Information Systems Engineering & Management, 2026. Disponível em: <https://doi.org/10.55267/iadt.07.2026.20>. Acesso em: 08 ago. 2026.

NAV IT. Adopting GitHub Copilot in a Large Public Sector Organization: A Longitudinal Study. 2025. Disponível em: <https://arxiv.org/abs/2509.20353>. Acesso em: 08 ago. 2026.

OFLIGHT. Codex vs Claude Code vs Cursor vs Copilot: 2026 AI Coding Tool Comparison. 2026. Disponível em: <https://www.oflight.co.jp/en/columns/codex-vs-claude-code-cursor-copilot-comparison-2026>. Acesso em: 08 ago. 2026.

SABRA, Abbas; SCHMITT, Olivier; SONAR, Joseph Tyler. Assessing the Quality and Security of AI-Generated Code: A Quantitative Analysis. 2025. Disponível em: <https://arxiv.org/abs/2508.14727>. Acesso em: 08 ago. 2026.

SILICONFLOW. The Best Open Source LLM for Engineering in 2026. 2026. Disponível em: <https://www.siliconflow.com/articles/en/best-open-source-LLM-for-engineering>. Acesso em: 08 ago. 2026.

SNYK. AI Code Generation: Code Security & Quality, Benefits, Risks & Tools. 2025. Disponível em: <https://snyk.io/blog/ai-code-generation-code-security-quality-benefits-risks-top-tools/>. Acesso em: 08 ago. 2026.

STAHNKE, Eduardo; VAHLICK, Adilson. Inteligência Artificial Aplicada na Engenharia de Software. Resumos Internos, v. 2, n. 1, 2013. Disponível em: <https://www.researchgate.net/publication/392212068>. Acesso em: 08 ago. 2026.

STRAY, Viktoria; MOE, Nils Brede; GANESHAN, Nina; KOBENES, Sebastian. Generative AI and Developer Workflows: How GitHub Copilot and ChatGPT Influence Solo and Pair Programming. 2025. Disponível em: <https://arxiv.org/abs/2503.12131>. Acesso em: 08 ago. 2026.

TEQNOVOS. Top Trends in Large Language Models (LLMs) for Software Development. 2025. Disponível em: <https://teqnovos.com/blog/top-trends-in-large-language-models-llms-for-software-development-in-2026/>. Acesso em: 08 ago. 2026.

TERRAGNI, Valerio; ROOP, Partha; BLINCOE, Kelly. The Future of Software Engineering in an AI-Driven World. 2024. Disponível em: <https://arxiv.org/abs/2406.07737>. Acesso em: 08 ago. 2026.

TOOLBOXKART. OpenAI Codex vs GitHub Copilot vs Claude Code (2026). 2026. Disponível em: <https://toolboxkart.tech/blog/codex-vs-github-copilot-vs-claude-code/>. Acesso em: 08 ago. 2026.

ZHU, Yuecai; TSANTALIS, Nikolaos; RIGBY, Peter C. AI-Generated Smells: An Analysis of Code and Architecture in LLM- and Agent-Driven Development. 2026. Disponível em: <https://arxiv.org/abs/2605.02741>. Acesso em: 08 ago. 2026.